

SCREENSHOTS OF SUBMISSIONS

Problem	Language	Time	Result	Score	Action
Diagonal Difference	c	12 minutes	Accepted	10.0	<button>View results</button>
Sparse Arrays	c	1 day	Accepted	25.0	<button>View results</button>
Compare the Triplets	c	1 day	Accepted	10.0	<button>View results</button>
Time Conversion	c	1 day	Accepted	15.0	<button>View results</button>
Diagonal Difference	c	1 day	Accepted	10.0	<button>View results</button>
Dynamic Array	c	1 day	Accepted	15.0	<button>View results</button>

Diagonal Difference ★

Problem

Submissions

Leaderboard

Discussions

Editorial

You made this submission 14 minutes ago.

Score: 10.00 Status: **Accepted**

People who solved **Diagonal Difference** attempted this next:

Prepare > Data Structures > Arrays > Sparse Arrays

Sparse Arrays ★

Problem

Submissions

Leaderboard

Discussions

Editorial

You made this submission 2 days ago.

Score: 25.00 Status: **Accepted**

People who solved **Sparse Arrays** attempted this next:

Compare the Triplets ★

Problem

Submissions

Leaderboard

Discussions

Editorial

You made this submission 2 days ago.

Score: 10.00 **Status:** Accepted

People who solved **Compare the Triplets** attempted this next:

Dynamic Array ★

Problem

Submissions

Leaderboard

Discussions

Editorial

You made this submission 2 days ago.

Score: 15.00 **Status:** Accepted

People who solved **Dynamic Array** attempted this next:

Time Conversion ★

Problem

Submissions

Leaderboard

Discussions

Editorial

You made this submission 2 days ago.

Score: 15.00 **Status:** Accepted

People who solved **Time Conversion** attempted this next:

GITHUB REPOSITORY LINK :

[harshavardhannayaka20072007-png/HackerRank-3rdSem-Portfolio](https://github.com/harshavardhannayaka20072007-png/HackerRank-3rdSem-Portfolio/blob/main/README.md)

The screenshot shows a GitHub repository page for 'HackerRank-3rdSem-Portfolio' by user 'harshavardhannayaka20072007-png'. The repository contains C implementations of selected HackerRank practice problems. The README file is displayed, showing a title 'HackerRank Practice Solutions in C', a description, and a list of challenges. The challenges are: 1. 01_Diagonal_Difference (Calculates the absolute difference between the sums of the two diagonals in a matrix.), 2. 02_Dynamic_Array (Implements dynamic sequences using XOR-indexed access and dynamic storage.), 3. 03_Time_Conversion (Converts time from 12-hour format to 24-hour format.), 4. 04_Compare_The_Triplets (Compares two triplets and returns the resulting score.), and 5. 05_Sparse_Arrays (Counts how many times each query string appears in the input strings.). The repository also includes a 'Notes' section.

HACKER RANK PROFILE URL :

[TS HARSHAVARDHAN NAYAKA - harshavardhann35 | HackerRank](#)

The screenshot shows a HackerRank profile page for 'TS HARSHAVARDHAN NAYAKA' (@harshavardhann35). The profile is currently incomplete, with a progress bar showing 0%. The user's personal information is listed: email (harshavardhannayaka20072007@gmail.com), mobile number (Add your mobile number), and location (India). The user's resume is also listed, with a link to '+ Add Resume'. The profile includes a section for 'Orchestrate' (a contest series) and a section for 'My Badges' (showing a 'Problem Solving' badge). The user's profile is currently incomplete, with a progress bar showing 0%.

Reflective Summary: Algorithmic Optimization & Memory Management

Completing these five HackerRank challenges significantly enhanced my understanding of algorithmic efficiency, data structures, and memory management in C.

First, **loop consolidation and linear traversal**—applied in *Diagonal Difference*, *Compare the Triplets*, and *Time Conversion*—taught me how to reduce complex workflows into clean $O(n)$ single-pass implementations. By merging parallel evaluation logic, I successfully minimized redundant CPU cycles and improved execution flow.

Second, the *Dynamic Array* problem underscored the critical role of **efficient memory scaling**. Utilizing `malloc`, `calloc`, and precise `realloc` operations taught me how to handle variable-length data structures dynamically. This minimized memory waste while preventing runtime errors like segmentation faults and memory leaks through proper cleanup (`free`).

Finally, challenges like *Sparse Arrays* highlighted the importance of **optimizing data lookup strategies**. Streamlining indexing and string-matching logic drastically improved performance when processing large datasets compared to brute-force alternatives.

Ultimately, these exercises bridged theoretical concepts with practical implementation, teaching me to carefully balance execution speed, algorithmic complexity, and memory economy in systems programming.